Rekursion

Programmieren und Software-Engineering
Theorie

9. Februar 2026

Definition

Definition (Rekursion)

Unter einer rekursiven Methode versteht man eine Methode die sich selbst aufruft.

- Die Definition gilt auch für *Funktionen* oder *Programme* (statt Methoden).
- Viele Problemstellungen/Algorithmen lassen sich sehr einfach und elegant mittels Rekursion formulieren.
- Abbruchbedingung: muss vorhanden sein um die rekursiven Aufrufe zu beenden.

Fakultät

In der Mathematik bezeichnet $n!$ die Fakultät, und es gilt

$$n! = n \cdot (n-1) \cdot \ldots \cdot 2 \cdot 1$$

für alle $n > 0$ und $0! = 1$.

Rekursive Formulierung der Fakultät:

$$n! = n \cdot (n-1)! \qquad \text{für } n > 0$$

$$0! = 1$$

Direkte Umsetzung in Programmcode:

```
long factorial(long n) {
    if (n == 0) return 1;
    return n * factorial(n-1);
}
```

Fakultät

In der Mathematik bezeichnet $n!$ die Fakultät, und es gilt

$$n! = n \cdot (n-1) \cdot \ldots \cdot 2 \cdot 1$$

für alle $n > 0$ und $0! = 1$.

Rekursive Formulierung der Fakultät:

$$n! = n \cdot (n-1)! \qquad \text{für } n > 0$$

$$0! = 1$$

Direkte Umsetzung in Programmcode:

```
1    long factorial(long n) {
2        if (n == 0) return 1;
3        return n * factorial(n-1);
4    }
```

Achtung: der Aufruf `factorial(-1)` führt zu einer endlosen Rekursion! Dieser Fall sollte im Code abgefangen werden.

Negativbeispiel: Fibonacci-Zahlen

Die Folge der Fibonacci-Zahlen ist für $n \geq 2$ definiert als

$$f_n = f_{n-1} + f_{n-2},$$

weiters gilt $f_0 = f_1 = 1$.

Hierdurch erhalten wir die Folge:

$$1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233, 377, 610, \ldots$$

Negativbeispiel: Fibonacci-Zahlen

Die direkte Umsetzung als rekursives Programm ist jedoch sehr unvorteilhaft:

```
long fibonacci(long n) {
    if (n <= 1) return 1;
    return fibonacci(n-1) + fibonacci(n-2);
}
```

Negativbeispiel: Fibonacci-Zahlen

Die direkte Umsetzung als rekursives Programm ist jedoch sehr unvorteilhaft:

```
long fibonacci(long n) {
    if (n <= 1) return 1;
    return fibonacci(n-1) + fibonacci(n-2);
}
```

Frage

Wo genau liegt das Problem bei der rekursiven Implementierung der Fibonacci-Zahlen?

Negativbeispiel: Fibonacci-Zahlen

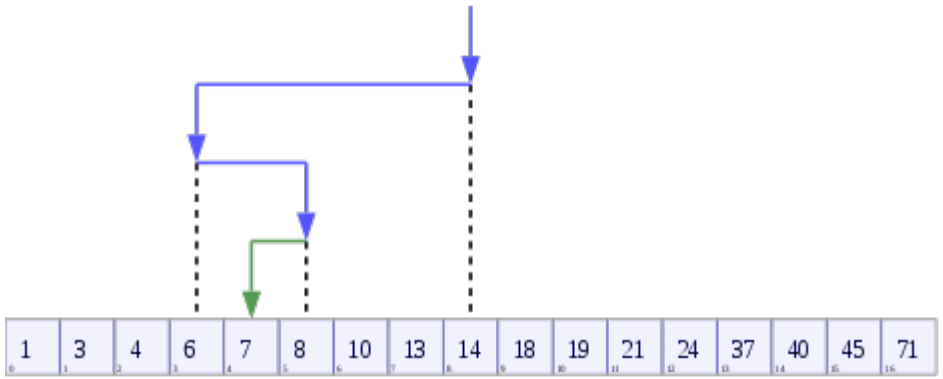
Die *iterative* Implementierung ist hier vorteilhaft, da wesentlich weniger Berechnungen ausgeführte werden:

```
long fibonacci(long n) {
    if (n < 0 || n > MAX) {
        // Ausnahmebehandlung
    }
    long[] f = new long[n+1];
    f[0] = 1; f[1] = 1;
    for (long i=2; i<=n; i++) {
        f[i] = f[i-1] + f[i-2];
    }
    return f[n];
}
```

Anmerkung: Die Berechnung des n-ten Folgegliedes ist auch ohne Array möglich!

Suche in sortierten Listen

- Aufgabenstellung: Suchen eines Elementes in einer sortierten Liste (Array)
- Naiver Zugang: $O(n)$ Schritte (Erwartungswert $n/2$)
- Mittels binärer Suche: nur $O(\log n)$ Schritte notwendig
- Vorgehensweise: analog zu Suche in Telefonbuch
  - Beliebige Seite aufschlagen
  - Steht Name davor oder danach?
  - Weitere Suche erfolgt nur mehr im verbleibenden Teil



Binäre Suche

Algorithmus 1: (Iterative) Binäre Suche

```
Function BinarySearch(sorted array a[], element e)
    Result: found element, or NULL if element not
            exists
    l = 0;
    r = a.length;
    while l ≤ r do
        m = l + (r−l)/2;
        if (a[m] == e) then
            return e;
        else
            if a[m] > e then
                r = m − 1;
            else
                l = m + 1;
    return NULL;
```

Rekursive Binäre Suche

Algorithmus 2: Rekursive Binäre Suche

```
Function RecBinarySearch(sorted array a[], element e, l, r)
    Result: found element, or NULL if element not exists
    if l ≤ r then
        m = l + (r−l)/2;
        if (a[m] == e) then
            return e;
        else
            if a[m] > e then
                return RecBinarySearch(a, e, m+1, r);
            else
                return RecBinarySearch(a, e, l, m-1);
    else
        return NULL;
```

Aufruf mit array a und gesuchtem Element e und `RecBinarySearch(a, e, 0, a.length-1)`.

Rekursive Binäre Suche

Eine rekursive Variante in Python:

```
1  def binaersuche_rekursiv(werte, gesucht, start, ende):
2      if ende < start:
3          return 'nicht␣gefunden'
4          # alternativ: return -start # bei (-Returnwert) waere
5          # die richtige Einfuege-Position
6
7      mitte = (start + ende) // 2
8      if werte[mitte] == gesucht:
9          return mitte
10     elif werte[mitte] < gesucht:
11         return binaersuche_rekursiv(werte, gesucht, mitte+1, ende)
12     else:
13         return binaersuche_rekursiv(werte, gesucht, start, mitte-1)
14
15 def binaersuche(werte, gesucht):
16     return binaersuche_rekursiv(werte, gesucht, 0, len(werte)-1)
```

Quelle: Wikipedia

Tiefensuche

Bei der Tiefensuche ist eine rekursive Implementierung naheliegend. Diese Variante durchläuft alle vom ersten Aufruf mit Knoten $v \in V(G)$ (`RecDFS(v)`) aus erreichbaren Knoten des Graphen G .

Algorithmus 3: Rekursive Tiefensuche

```
1 Function RecDFS(Knoten v)
2     markiere v als besucht;
3     for all [v, u] ∈ E(G) do
4         if u noch nicht besucht then
5             RecDFS(u);
```

In der rekursiven Version der Tiefensuche ist kein *Stack* notwendig, die Knoten werden durch die rekursiven Aufrufe automatisch in der richtigen Reihenfolge besucht.

Tiefensuche

Diese Variante sucht einen Knoten s und returniert ihn sobald gefunden:

Algorithmus 4: Rekursive Tiefensuche (2)

```
1 Function RecDFS(Knoten v, Gesuchter Knoten s)
2     if v gesuchter Knoten s then
3         return v;
4     markiere v als besucht;
5     for all [v, u] ∈ E(G) do
6         if u noch nicht besucht then
7             if RecDFS(u, s) == s then
8                 return s;
9     return null;
```

Programmierbeispiel 12.1.1

Gegeben ist ein 2-dimensionales Boolean-Array, befüllt mit *wahr* und *falsch*. Ermitteln Sie mit einem Programm die Größe des größten zusammenhängenden Gebietes mit Wert wahr.

Beispiel: hier wird falsch mit einem '.' und wahr mit einem 'X' dargestellt.

```
...XXX..XX..
.XXX.XXXX...
X.XX..XX..XX
X...XXX..XXX
XXX....XXX..
XX.XX..X..XX
...X..X.XXX.
.....XXXX...
```

In diesem Beispiel ist die Größe 19.

Zusatzaufgabe 12.1.2

Füllen Sie ein zweidimensionales Boolean-Array (Größe 300x300) mit Zufallswerten, wobei mit einer Wahrscheinlichkeit von $1/3$ der Wert wahr, und mit $2/3$ der Wert falsch gesetzt werden soll. Wenden Sie den Algorithmus aus dem vorigen Beispiel an. Ist das Ergebnis plausibel? Was passiert wenn man die Wahrscheinlichkeiten tauscht?

Programmierbeispiel 12.2.1

Implementieren Sie Mergesort in Java.

Programmierbeispiel 12.2.2

Implementieren Sie Quicksort in Java.

```
private static boolean[][] werte = {
                 {false,false,false,true,true,true,false,false,true,true,false,false},
                 {false,true,true,true,false,true,true,true,true,false,false,false},
                 {true,false,true,true,false,false,true,true,false,false,true,true},
                 {true,false,false,false,true,true,true,false,false,true,true,true},
                 {true,true,true,false,false,false,false,true,true,true,false,false},
                 {true,true,false,true,true,false,false,true,false,false,true,true},
                 {false,false,false,true,false,false,true,false,true,true,true,false},
                 {false,false,false,false,false,true,true,true,true,false,false,false}
                 };
```